

Constraints in SQL

SQL constraints are rules enforced on data columns in a table. They are used to limit the type of data that can go into a table, ensuring the accuracy and reliability of the data in the database. Constraints can be column-level or table-level. Column-level constraints apply to a single column, while table-level constraints apply to the whole table.

Types of Constraints

There are several types of constraints in SQL, each serving a specific purpose in maintaining data integrity.

Constraint Type	Description
NOT NULL	Ensures that a column cannot have a NULL value.
UNIQUE	Ensures that all values in a column are different.
PRIMARY KEY	Uniquely identifies each record in a table. It cannot contain NULL values.
FOREIGN KEY	Uniquely identifies a record in another table. It establishes a link between two tables.
CHECK	Ensures that all values in a column satisfy a specific condition.
DEFAULT	Sets a default value for a column when no value is specified.
INDEX	Used to create and retrieve data from the database very quickly.

NOT NULL Constraint

The **NOT NULL** constraint ensures that a column cannot have a **NULL** value. This means that you must provide a value for that column when inserting new records or updating existing ones.

Example: Creating a Table with **NOT NULL**

```
CREATE TABLE Employees (  
  
    EmployeeID INT NOT NULL,  
  
    FirstName VARCHAR(50) NOT NULL,  
  
    LastName VARCHAR(50) NOT NULL,  
  
    Email VARCHAR(100)  
  
);
```

Example: Inserting Data into **Employees** Table

EmployeeID	FirstName	LastName	Email
101	John	Doe	john.doe@example.com
102	Jane	Smith	jane.smith@example.com
103	Peter	Jones	peter.jones@example.com

UNIQUE Constraint

The **UNIQUE** constraint ensures that all values in a column are different. While a **PRIMARY KEY** constraint automatically has a **UNIQUE** constraint, you can have many **UNIQUE** constraints in a table, but only one **PRIMARY KEY** constraint.

Example: Creating a Table with **UNIQUE**

```
CREATE TABLE Products (  
  
    ProductID INT NOT NULL UNIQUE,
```

```
    ProductName VARCHAR(255) NOT NULL,  
  
    SKU VARCHAR(50) UNIQUE  
  
);
```

Example: Inserting Data into **Products** Table

ProductID	ProductName	SKU
1	Laptop	LP001
2	Mouse	MS002
3	Keyboard	KB003

PRIMARY KEY Constraint

The **PRIMARY KEY** constraint uniquely identifies each record in a table. Primary keys must contain **UNIQUE** values, and cannot contain **NULL** values. Each table can have only one primary key, which can consist of single or multiple columns.

Example: Creating a Table with **PRIMARY KEY**

```
CREATE TABLE Orders (  
  
    OrderID INT PRIMARY KEY,  
  
    CustomerID INT,  
  
    OrderDate DATE  
  
);
```

Example: Inserting Data into **Orders** Table

OrderID	CustomerID	OrderDate
1001	1	 Date
1002	2	 Date
1003	1	 Date

FOREIGN KEY Constraint

A **FOREIGN KEY** is a field (or collection of fields) in one table that refers to the **PRIMARY KEY** in another table. The table containing the foreign key is called the child table, and the table containing the primary key is called the referenced or parent table.

Example: Creating Tables with **FOREIGN KEY**

```
CREATE TABLE Customers (
```

```
    CustomerID INT PRIMARY KEY,
```

```
    FirstName VARCHAR(50),
```

```
    LastName VARCHAR(50)
```

```
);
```

```
CREATE TABLE Orders (
```

```
    OrderID INT PRIMARY KEY,
```

```
    CustomerID INT,
```

```
    OrderDate DATE,
```

```
    FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID)
```

```
);
```

Example: Data in **Customers** and **Orders** Tables

Customers Table:

CustomerID	FirstName	LastName
1	Alice	Johnson
2	Bob	Williams

Orders Table:

OrderID	CustomerID	OrderDate
1001	1	 Date
1002	2	 Date

CHECK Constraint

The **CHECK** constraint is used to limit the value range that can be placed in a column. If you define a **CHECK** constraint on a column, it will allow only certain values for this column.

Example: Creating a Table with **CHECK**

```
CREATE TABLE Persons (
```

```
    PersonID INT PRIMARY KEY,
```

```
    LastName VARCHAR(255) NOT NULL,
```

```
    FirstName VARCHAR(255),
```

```
    Age INT,
```

```
    CHECK (Age >= 18)
```

```
);
```

Example: Inserting Data into **Persons** Table

PersonID	LastName	FirstName	Age
1	Doe	John	30
2	Smith	Jane	25
3	Williams	Bob	18

DEFAULT Constraint

The **DEFAULT** constraint is used to set a default value for a column. The default value will be added to all new records if no other value is specified.

Example: Creating a Table with **DEFAULT**

```
CREATE TABLE ProductsInventory (
```

```
    ProductID INT PRIMARY KEY,
```

```
    ProductName VARCHAR(255),
```

```
    Price DECIMAL(10, 2),
```

```
    StockQuantity INT DEFAULT 0
```

```
);
```

Example: Inserting Data into **ProductsInventory** Table

ProductID	ProductName	Price	StockQuantity
1	Monitor	299.99	100
2	Webcam	49.99	50
3	Speakers	75.00	0

INDEX Constraint

An **INDEX** is used to retrieve data from the database more quickly than otherwise. The users cannot see the indexes; they are just used to speed up searches/queries.

Example: Creating an **INDEX**

```
CREATE INDEX idx_lastname
```

```
ON Employees (LastName);
```

This index on the **LastName** column in the **Employees** table will significantly speed up queries that filter or sort by employee last names.